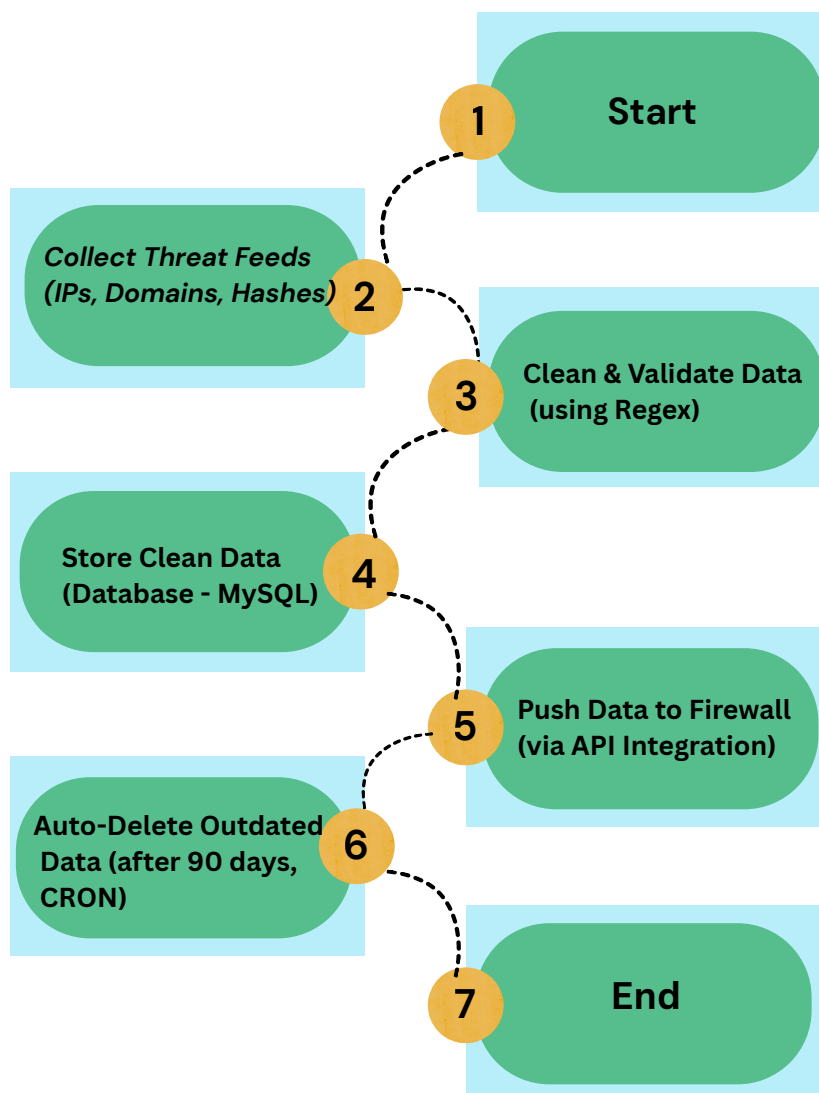


2- Problem statement

In today's time, a huge amount of threat intelligence data such as IP addresses, domains, and file hashes is generated every day. Handling this data manually is difficult, time-consuming, and can also lead to mistakes. If outdated or duplicate data is not removed on time, it may reduce the efficiency of security systems and affect the firewall's performance. Therefore, there is a need for a system that can automatically clean, manage, and update threat feeds in the firewall without human effort.

3- Solution Flowchart



4- Objective / Proposed Solution

4.1- Threat Feed cleaning (Regex based)

4.1.1 - IPv4 Address Extraction

Step 1- Raw Data

Input may be obfuscated, e.g., [.] (dot).
for ex- 1.2.3.4 , 34[.]45[.]34[.]56

Step 2- Initial Checks

- String contains digits and dot-like characters → candidate for IPv4.
- Has exactly 3 dot separators.
- Each part (octet) is numeric.

Step 3- Normalization

Replace obfuscations with real . →
34[.]45[.]34[.]56 → 34.45.34.56

Step 4- Regex Pattern

`\b(?:25[0-5]|2[0-4]\d|1?\d?\d)(?:\.(?:25[0-5]|2[0-4]\d|1?\d?\d)){3}\b`

Step 5- Regex Working

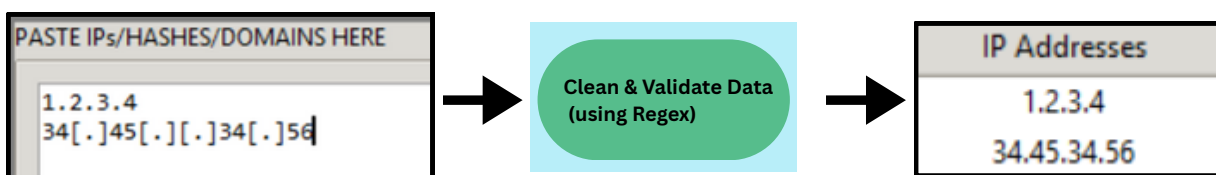
- 25[0-5] → captures numbers 250–255.
- 2[0-4]\d → captures numbers 200–249.
- 1?\d?\d → captures numbers 0–199.
- (...) {3} → repeats the octet pattern 3 more times with dots.

Step 6- Example Walkthrough

- Input: 34.45.34.56
- First octet: 34 → matches 1?\d?\d (valid).
- Second octet: 45 → matches 1?\d?\d (valid).
- Third octet: 34 → matches 1?\d?\d (valid).
- Fourth octet: 56 → matches 1?\d?\d (valid).
- ☒ Final Output → 34.45.34.56 is a valid IPv4.

Step 7- Post-Processing

- Ensure all octets are 0–255.
- Exclude private IP ranges like 192.168.x.x.
- Return clean dotted-decimal IP.



4.1.2 - Domain Extraction

Step 1- Raw Data

Domains often appear obfuscated.

for ex- `www[.]example[.]com` , `test(dot)org`

Step 2- Initial Checks

- Contains at least one dot.
- Each part (label) is alphanumeric or -.
- Last part (TLD) must have ≥ 2 letters.

Step 3- Normalization

Replace obfuscations with real . $\rightarrow$

`www[.]example[.]com` $\rightarrow$ `www.example.com`

`test(dot)org` $\rightarrow$ `test.org`

Step 4- Regex Pattern

`\b((?:[a-zA-Z0-9-]+\.)+[a-zA-Z]{2,})\b`

Step 5- Regex Working

- `[a-zA-Z0-9-]+` $\rightarrow$ valid domain/subdomain.
- `\.` $\rightarrow$ dot separator.
- `[a-zA-Z]{2,}` $\rightarrow$ valid TLD (e.g., com, org).

Step 6- Example Walkthrough

- Input: `www.example.com`
 - First part: `www` $\rightarrow$ matches `[a-zA-Z0-9-]+`
 - Dot $\rightarrow$ matches `\.`
 - Second part: `example` $\rightarrow$ matches `[a-zA-Z0-9-]+`
 - Dot $\rightarrow$ matches `\.`
 - TLD: `com` $\rightarrow$ matches `[a-zA-Z]{2,}`
 - ☒ Final Output $\rightarrow$ `www.example.com` is a valid domain.
- Input: `test.org`
 - `test` matches first part, `.org` matches TLD.
 - ☒ Final Output $\rightarrow$ `test.org` is a valid domain.

Step 7- Post-Processing

- Convert to lowercase.
- Cross-check against blacklist/whitelist.



4.1.3 - Hash Extraction

Step 1- Raw Data

Long suspicious alphanumeric strings.

Step 2- Initial Checks

- Contains only hex chars [0-9a-fA-F].
- Length matches known hash sizes:

Step 3- Regex Pattern

- MD5 → `\b[a-fA-F0-9]{32}\b`
- SHA1 → `\b[a-fA-F0-9]{40}\b`
- SHA256 → `\b[a-fA-F0-9]{64}\b`

For All hash - `r'\b[a-fA-F0-9]{16,64}\b'`

Step 5- Regex Working

- `[a-fA-F0-9]` → only hex characters allowed.
- `{16,64}` → between 16 and 64 characters.

Step 6- Example Walkthrough

- Input: 098f6bcd4621d373cade4e832627b4f6
 - Length = 32 → between.
 - All characters are hex → valid.
 - ☒ Final Output → Detected as hash.
- Input: 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
 - Length = 40 → between 16 and 64.
 - All characters hex → valid.
 - ☒ Final Output → Detected as hash.

Step 7- Post-Processing

- Extract all hash from all given values

PASTE IPs/HASHES/DOMAINS HERE
098f6bcd4621d373cade4e832627b4f6 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8



Hashes
098f6bcd4621d373cade4e832627b4f6 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8

4.2- Database Storage

4.2.1 - Reason for Choosing MySQL

- Well-structured and reliable for relational data.
- Supports indexing for fast lookups of large IOC datasets.
- Widely supported, open-source, and integrates easily with Python/PHP backend.

4.2.2 - Database Schema

- The main table are
(fig no 4.2.1) :-
 1. domains
 2. hashes
 3. ips

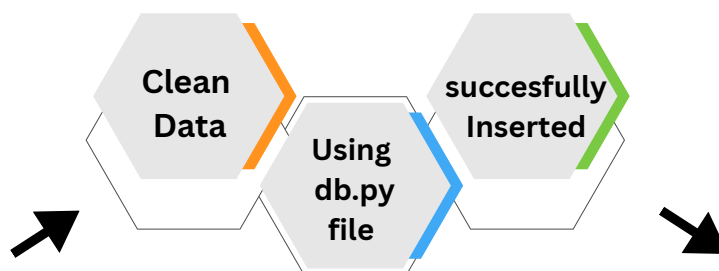
Fig no - 4.2.1

```
mysql> desc domains;
+-----+-----+-----+-----+-----+-----+
| Field | Type   | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| id     | int    | NO   | PRI | NULL    | auto_increment |
| domain | varchar(255) | YES |     | NULL    |                 |
| timestamp | datetime | YES |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)

mysql> desc hashes;
+-----+-----+-----+-----+-----+-----+
| Field | Type   | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| id     | int    | NO   | PRI | NULL    | auto_increment |
| hash   | text   | YES  |     | NULL    |                 |
| timestamp | datetime | YES |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> desc ips;
+-----+-----+-----+-----+-----+-----+
| Field | Type   | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| id     | int    | NO   | PRI | NULL    | auto_increment |
| ip     | varchar(45) | YES |     | NULL    |                 |
| timestamp | datetime | YES |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

4.2.3 - Clean Data Insertion In Database



IP Addresses
IP Addresses
34.45.34.56
1.2.3.4

Domains
Domains
test.org
www.example.com

Hashes
Hashes
098f6bcd4621d373cade4e832627b4f6
5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8

```
mysql> select * from ips;
+----+-----+-----+
| id | ip       | timestamp                |
+----+-----+-----+
| 45 | 34.45.34.56 | 2025-09-08 12:23:03 |
| 46 | 1.2.3.4   | 2025-09-08 12:23:03 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> select * from domains;
+----+-----+-----+
| id | domain      | timestamp                |
+----+-----+-----+
| 35 | test.org    | 2025-09-08 12:23:03 |
| 36 | www.example.com | 2025-09-08 12:23:03 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> select * from hashes;
+----+-----+-----+
| id | hash                                               | timestamp                |
+----+-----+-----+
| 36 | 098f6bcd4621d373cade4e832627b4f6                | 2025-09-08 12:23:03 |
| 37 | 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8        | 2025-09-08 12:23:03 |
+----+-----+-----+
2 rows in set (0.00 sec)
```

4.3- Api pusher

4.3.1 - Data Fetching

- The module fetches the cleaned IOCs (IPs, domains, hashes) from the MySQL database.

4.3.2 - Formatting

- Converts the data into the required JSON structure accepted by the mock firewall API.

```
json={"ip": ip})
```

```
json={"domain": domain})
```

```
json={"hash": h})
```

4.3.3 - API Request

- Uses HTTP POST request to push data into the mock firewall.



```
def push_ips(ip_list):
    for ip in ip_list:
        try:
            response = requests.post("http://localhost/api/ip_push", json={"ip": ip})
            print(f"IP PUSH: {ip} → {response.status_code}")
        except Exception as e:
            print(f"Error pushing IP {ip}: {e}")
```



```
def push_domains(domain_list):
    for domain in domain_list:
        try:
            response = requests.post("http://localhost/api/domain_push", json={"domain": domain})
            print(f"DOMAIN PUSH: {domain} → {response.status_code}")
        except Exception as e:
            print(f"Error pushing Domain {domain}: {e}")
```



```
def push_hashes(hash_list):
    for h in hash_list:
        try:
            response = requests.post("http://localhost/api/hash_push", json={"hash": h})
            print(f"HASH PUSH: {h} → {response.status_code}")
        except Exception as e:
            print(f"Error pushing Hash {h}: {e}")
```

4.3.4 - Response Handling

- Confirms success or failure of each push based on API response (e.g., status code 200 OK). (Fig no - 4.3.4)

```
🔥 [IP PUSH] → 1.2.3.4
127.0.0.1 - - [08/Sep/2025 21:59:20] "POST /api/ip_push HTTP/1.1" 200 -
🔥 [IP PUSH] → 34.45.34.56
127.0.0.1 - - [08/Sep/2025 21:59:22] "POST /api/ip_push HTTP/1.1" 200 -
🌐 [DOMAIN PUSH] → www.example.com
127.0.0.1 - - [08/Sep/2025 21:59:24] "POST /api/domain_push HTTP/1.1" 200 -
🌐 [DOMAIN PUSH] → test.org
127.0.0.1 - - [08/Sep/2025 21:59:26] "POST /api/domain_push HTTP/1.1" 200 -
🔒 [HASH PUSH] → 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
127.0.0.1 - - [08/Sep/2025 21:59:29] "POST /api/hash_push HTTP/1.1" 200 -
🔒 [HASH PUSH] → 098f6bcd4621d373cade4e832627b4f6
127.0.0.1 - - [08/Sep/2025 21:59:31] "POST /api/hash_push HTTP/1.1" 200 -
```

Fig no - 4.3.4

4.4- Auto Deletion

4.4.1 - IOC Expiry Assignment

- When an IP, domain, or hash is inserted into the database, a valid_until timestamp is automatically set (default: 90 days from insertion).

4.4.2 - Scheduled Cleanup

- A cron job (or scheduler) runs daily at 2:00 AM to check for expired IOCs.

4.4.3 - Scheduled Cleanup

- Expired entries are removed from the MySQL Cleaned_IOCs table using a delete query. (Fig no - 4.4.3)

```
def cleanup_job():
    print("🔧 Starting cleanup job...")

    # Delete old entries from DB
    for table in ['ips', 'domains', 'hashes']:
        old_records = get_old_records(table)
        if old_records:
            print(f"🗑️ Deleting {len(old_records)} old records from {table}")
            delete_old_records(table)
        else:
            print(f"✅ No old records in {table}")

    # Call delete APIs to sync firewall
    delete_old_from_firewall()

    print("✅ Cleanup job completed.")
```

Fig no - 4.4.3

4.4.4 - Firewall Synchronization

- The API Pusher module also sends a delete request to the mock firewall, ensuring that the firewall rules stay updated and do not contain outdated indicators.(Fig no - 4.4.4)

```
# 🗑️ DELETE APIs - This runs automatically every night at 2 AM via Task Scheduler
Tabnine | Edit | Test | Explain | Document
def delete_old_from_firewall():
    for table, endpoint in [
        ("ips", "http://localhost/api/ip_delete"),
        ("domains", "http://localhost/api/domain_delete"),
        ("hashes", "http://localhost/api/hash_delete")
    ]:
        try:
            old_data = get_old_records(table)
            for record in old_data:
                value = record[1] # Assuming 2nd column is value (ip, domain, hash)
                response = requests.post(endpoint, json={"value": value})
                print(f"{table.upper()} DELETE: {value} → {response.status_code}")
        except Exception as e:
            print(f"Error deleting from {table}: {e}")

if __name__ == "__main__":
    delete_old_from_firewall()
```

Fig no - 4.4.4

5- Technology Stack

5.1- Programming Language

- **Python 3.x** → Core development language used for GUI, data cleaning, database interaction, and API integration.
- **MySQL** → (Relational Database for storing IOCs)

5.2- Frontend (GUI Layer)

- **Tkinter** → Used for creating the desktop GUI interface.
- **ttk (Themed Tkinter Widgets)** → For styled buttons, labels, and inputs.
- **ScrolledText** → For displaying logs and cleaned output in a scrollable text box.
- **MessageBox** → For user notifications and alerts.

5.3- Backend (Processing & Business Logic)

- **Regex** → To clean and validate IPs, domains, and hashes.
- **Threading** → To handle background tasks like pushing IOCs to API without freezing the GUI.
- **Datetime & Timedelta** → For timestamping and managing IOC expiry (auto-deletion after 90 days).

5.4- Database Layer

- **MySQL** → Relational database to store cleaned IOCs.
- **mysql.connector** → Python-MySQL connector library for database operations (insert, fetch, delete).

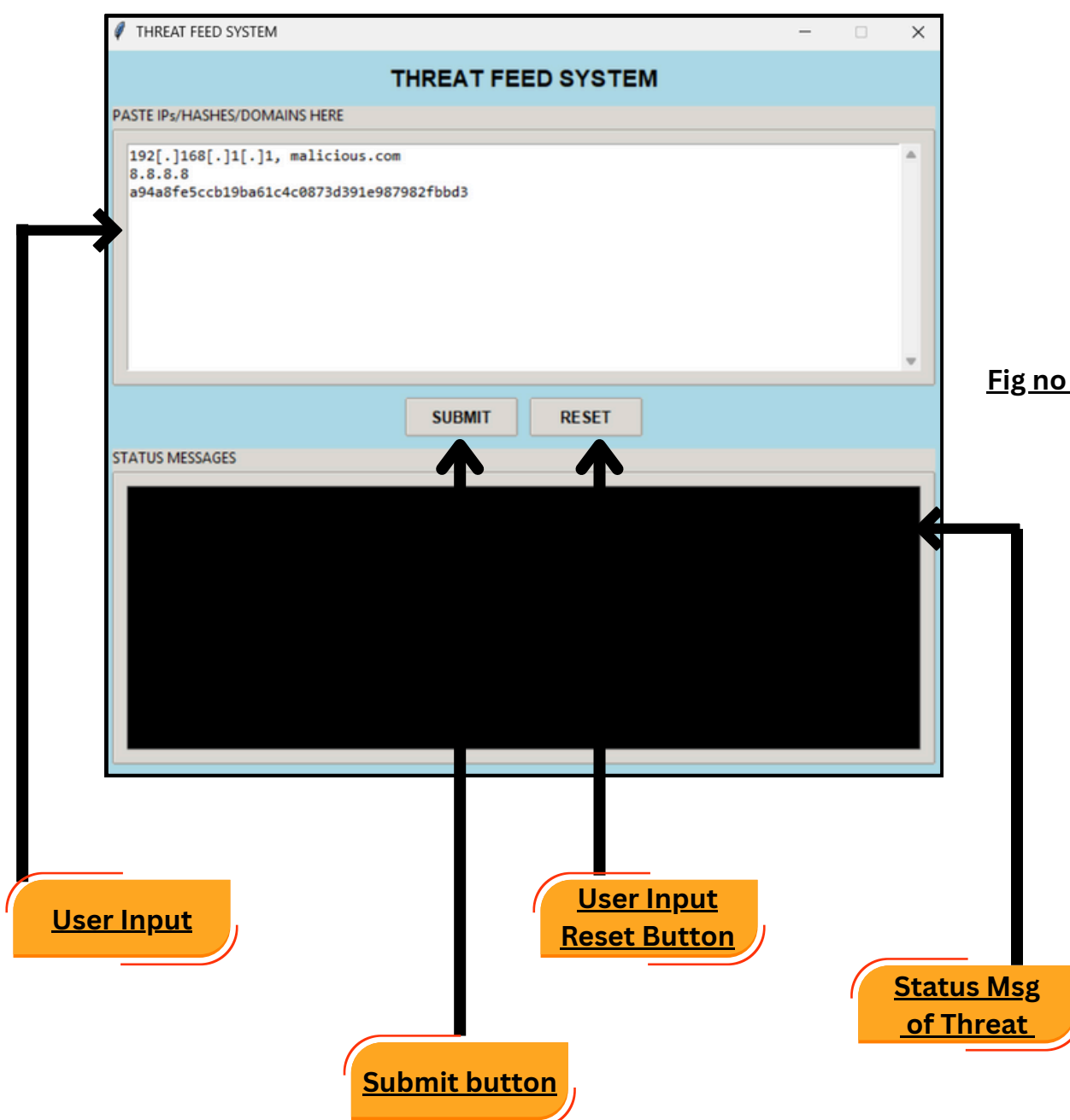
5.5- API Integration

- **Requests (Python library)** → To push cleaned IOCs from database to the mock firewall via REST API.
- **Flask (Python micro-framework)** → To create the mock firewall API endpoints for accepting IOC pushes and deletions.

6- Implementation Overview

6.1- interface.py Module → { User Input, Data Cleaning, and Interface }

- The Tkinter-based GUI allows the user to enter raw IOCs (IPs, domains, hashes). (Fig no - 6.1)
- Integrated regex-based logic inside this module cleans and validates the input.
- Only valid IOCs are displayed back to the user and forwarded for storage.



6.2- db.py → { Database Storage }

- The validated IOCs are stored in a MySQL database along with timestamps. (Fig no - 6.2a)
- This module handles insertion of new records, retrieval for processing, and deletion of expired IOCs. (Fig no - 6.2b)

```
mysql> desc domains;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id    | int       | NO   | PRI | NULL    | auto_increment |
| domain | varchar(255) | YES |     | NULL    |                |
| timestamp | datetime | YES |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)

mysql> desc hashes;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id    | int       | NO   | PRI | NULL    | auto_increment |
| hash  | text      | YES  |     | NULL    |                |
| timestamp | datetime | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> desc ips;
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id    | int       | NO   | PRI | NULL    | auto_increment |
| ip    | varchar(45) | YES |     | NULL    |                |
| timestamp | datetime | YES |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Domain table
with timestamp

Hash table
with timestamp

Ip table
with timestamp

Fig no - 6.2a

```
mysql> select * from ips;
+----+-----+-----+
| id | ip      | timestamp          |
+----+-----+-----+
| 45 | 34.45.34.56 | 2025-09-08 12:23:03 |
| 46 | 1.2.3.4   | 2025-09-08 12:23:03 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> select * from domains;
+----+-----+-----+
| id | domain          | timestamp          |
+----+-----+-----+
| 35 | test.org        | 2025-09-08 12:23:03 |
| 36 | www.example.com | 2025-09-08 12:23:03 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> select * from hashes;
+----+-----+-----+
| id | hash                                     | timestamp          |
+----+-----+-----+
| 36 | 098f6bcd4621d373cade4e832627b44f6   | 2025-09-08 12:23:03 |
| 37 | 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8 | 2025-09-08 12:23:03 |
+----+-----+-----+
2 rows in set (0.00 sec)
```

Insertion of new
Records In Tables

Fig no - 6.2b

6.3- api_pusher.py → { API Pushing }

- Fetches cleaned IOC's from the database and pushes them to the mock firewall via REST API calls. (Fig no - 6.3)
- Uses POST requests to insert new IOC's and DELETE requests to remove expired ones.
- Ensures that the firewall remains synchronized with the database.

```
🔥 [IP PUSH] → 1.2.3.4
127.0.0.1 - - [08/Sep/2025 21:59:20] "POST /api/ip_push HTTP/1.1" 200 -
🔥 [IP PUSH] → 34.45.34.56
127.0.0.1 - - [08/Sep/2025 21:59:22] "POST /api/ip_push HTTP/1.1" 200 -
🌐 [DOMAIN PUSH] → www.example.com
127.0.0.1 - - [08/Sep/2025 21:59:24] "POST /api/domain_push HTTP/1.1" 200 -
🌐 [DOMAIN PUSH] → test.org
127.0.0.1 - - [08/Sep/2025 21:59:26] "POST /api/domain_push HTTP/1.1" 200 -
🔒 [HASH PUSH] → 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
127.0.0.1 - - [08/Sep/2025 21:59:29] "POST /api/hash_push HTTP/1.1" 200 -
🔒 [HASH PUSH] → 098f6bcd4621d373cade4e832627b4f6
127.0.0.1 - - [08/Sep/2025 21:59:31] "POST /api/hash_push HTTP/1.1" 200 -
```

**Cleaned data push
to firewall**

Fig no - 6.3

6.4- mock_firewall.py → { Mock Firewall }

- A Flask-based API server simulating a real firewall environment. (Fig no - 6.4)
- Provides endpoints to accept IOC additions and deletions.
- Useful for testing API integration without requiring a real firewall.

```
* Serving Flask app 'mock_firewall'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://localhost:80
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 182-776-996
```

Fig no - 6.4

6.5- nightly_cleanup.py → { Automatic Cleanup }

- Scheduled to run daily at 2:00 AM. (Fig no - 6.5)
- Queries the database for expired IOC's (older than 90 days).
- Removes expired IOC's both from the database and the mock firewall through API calls.

```
# DELETE APIs - This runs automatically every night at 2 AM via Task Scheduler
def delete_old_from_firewall():
    for table, endpoint in [
        ("ips", "http://localhost/api/ip_delete"),
        ("domains", "http://localhost/api/domain_delete"),
        ("hashes", "http://localhost/api/hash_delete")
    ]:
        try:
            old_data = get_old_records(table)
            for record in old_data:
                value = record[1] # Assuming 2nd column is value (ip, domain, hash)
                response = requests.post(endpoint, json={"value": value})
                print(f"{table.upper()} DELETE: {value} + {response.status_code}")
        except Exception as e:
            print(f"Error deleting from {table}: {e}")

if __name__ == "__main__":
    delete_old_from_firewall()
```

**Code run at 2 AM
to delete Old
threat**

Fig no - 6.5

7- Result and benefits

7.1- ♦ Result

The project Threat Feed Automation via API successfully provides an end-to-end automated system for handling Indicators of Compromise (IOCs) like IPs, domains, and hashes.

- **User-friendly GUI** (Tkinter based) makes it easy to input threat data.
- **Data cleaning** ensures that only valid IOCs are processed.
- **MySQL database** stores all IOCs with timestamps for record keeping.
- **API Pusher** automatically sends cleaned data to the firewall through REST API.
- **Mock Firewall** simulates a real firewall, ensuring safe testing of API integration.
- **Nightly Cleanup** removes old IOCs (older than 90 days) automatically from both database and firewall.

This ensures the system is always updated with the latest threat intelligence and removes unnecessary/expired data without manual effort.

7.2- ♦ Benefits

- **Automation** – The whole process (cleaning, storing, pushing, and deleting) runs automatically, reducing manual work.
- **Accuracy** – Regex-based cleaning ensures only valid and correct IOCs are stored and pushed.
- **Centralized Storage** – MySQL database keeps all threat data in one place with timestamps, which helps in audits and analysis.
- **Firewall Synchronization** – The mock firewall always stays updated with the latest IOCs, making the system closer to real-world security operations.
- **Scalability** – The modular design allows easy extension (e.g., adding new IOC types or integrating with a real firewall).
- **Reliability** – Nightly cleanup prevents the system from being overloaded with outdated data, ensuring efficiency.
- **Practical Learning** – Demonstrates real-world cybersecurity practices such as API integration, database management, and scheduled automation.

8- Conclusion *and* Future Scope

8.1- ♦ Conclusion

The project “**Threat Feed Automation**” has been successfully implemented with a modular architecture combining GUI, data cleaning, database management, API integration, and automated cleanup. Through this system, raw Indicators of Compromise (IOCs) such as IP addresses, domains, and hashes are cleaned, validated, stored, and pushed to the firewall automatically.

The addition of the nightly cleanup process ensures that outdated IOCs are removed without manual intervention, keeping the system efficient and up-to-date. The project demonstrates the integration of multiple technologies including Python, MySQL, Tkinter, Regex, Flask API, and automation scheduling.

☞ In summary, this project provides a practical, reliable, and automated approach to threat feed management, which can be applied to real-world cybersecurity environments.

The most important achievement is the automation of IOC handling, which reduces manual effort and ensures real-time synchronization with the firewall.

8.2- ♦ Future Scope

- **Integration with Real Firewalls** – The mock firewall can be replaced with actual firewall solutions (e.g., Palo Alto, Fortinet, Cisco ASA) for real-world deployment.
- **Support for IPv6 and More IOC Types** – Current implementation focuses mainly on IPv4, domains, and hashes; support can be extended to IPv6 and URLs.
- **Machine Learning Integration** – AI/ML models can be added for detecting suspicious patterns and predicting malicious IOCs instead of only cleaning input.
- **Web-Based Dashboard** – A browser-based dashboard (with Flask/Django + React) can be developed to monitor IOCs in real time instead of using only Tkinter.
- **Threat Intelligence Sharing** – The system can be extended to pull data from open threat intelligence feeds (like AbuseIPDB, VirusTotal) and push them directly to the firewall.

9- Software Licensing

9.1- ♦ Types of Software Licenses

9.1.1 - Proprietary Software Licenses

➡ The copyright of the software belongs to the company. The user can only use it but cannot modify or redistribute it.

- **Commercial License**

1. Paid software.
2. A license must be purchased.
3. Example: Microsoft Office, Adobe Photoshop

- **Freeware**

1. Free to use.
2. Source code is not available.
3. Example: Skype, Zoom, Google Chrome

- **Shareware**

1. Provided in trial or demo form.
2. Limited time or limited features.
3. Later, the full version must be purchased.
4. Example: WinRAR, IDM (Internet Download Manager)

- **Closed Source License**

1. Code is hidden and controlled by the company.
2. Only executable software is provided.
3. Example: Windows OS

9.1.2 - Open-Source Software Licenses (FOSS)

➡ The source code is openly available. Users can modify and redistribute it, but each license has its own rules.

- **GNU General Public License (GPL)**

1. Copyleft license.
2. If you modify and distribute the software, the new version must also remain open-source.
3. Example: Linux Kernel, WordPress

- **MIT License**

1. Very permissive and flexible.
2. You can use, modify, and redistribute freely.
3. Only copyright notice is required.
4. Example: jQuery, Node.js

- **Apache License**
 1. Similar to MIT but also provides patent rights.
 2. Safe for corporate use.
 3. Example: Apache HTTP Server, Android
- **BSD License**
 1. Similar to MIT, very permissive.
 2. Simple with fewer restrictions.
 3. Example: FreeBSD OS
- **Mozilla Public License (MPL)**
 1. A middle-ground license.
 2. Modified files must remain open, but proprietary code can also be added.
 3. Example: Mozilla Firefox, Thunderbird

9.1.3 - Public Domain License

- No copyright exists on the software.
- Free to use, modify, and redistribute without restrictions.
- Example: SQLite database

9.1.4 - Creative Commons (CC) License

- Not mainly for software, but used for documents, media, and content.
- Has different versions (BY, SA, NC, ND).
- Example: Open educational content, images, articles

9.1.5 - EULA (End-User License Agreement)

- A contract for proprietary software.
- When you install the software, you must press "I Agree".
- Example: Windows, Adobe products

9.1.6 - Subscription-Based License

- Software can only be used as long as the subscription is active.
- Example: Microsoft 365, Adobe Creative Cloud, Spotify Premium

✓ Summary

- **Proprietary** → Control remains with the company (includes Paid, Freeware, Shareware).
- **Open-source** → Code is open for everyone (GPL, MIT, Apache, BSD, MPL).
- **Public Domain** → Completely free, no copyright.
- **Creative Commons** → Mostly for media, sometimes used for software documentation.
- **EULA** → Agreement between the company and the user.
- **Subscription** → Pay-as-you-use model.

9.2- ♦ License for Project

- For My project, the following three open-source licenses are considered as the most suitable:

1 MIT License

- It is very simple and widely adopted.
- Allows others to freely use, modify, and share the project.
- The only condition is giving proper credit to the original author.

👉 **Best because it keeps the project open and flexible for academic as well as professional use.**



2 Apache License 2.0

- Similar to MIT but more formal and enterprise-friendly.
- Provides explicit patent protection, making it suitable for organizations and companies.

👉 **Best because it makes the project look professional and future-ready for enterprise-level deployment.**



3 GNU GPL v3 License

- Strong copyleft license ensuring that the project and its derivatives always remain open-source.
- No one can legally convert it into a closed-source project.

👉 **Best because it protects the open-source nature of the project and encourages community contribution.**



✅ In summary, these three licenses (MIT, Apache 2.0, and GPL v3) are best choices as they ensure the project remains open-source, encourages collaboration, and provides flexibility for both academic and professional use.

10- References

- 1** Python Software Foundation, Python 3 Standard Library Documentation. Available at: <https://docs.python.org/3/library/>
- 2** MySQL Documentation, MySQL 8.0 Reference Manual. Oracle. Available at: <https://dev.mysql.com/doc/>
- 3** Flask Documentation, Flask Web Development Framework. Available at: <https://flask.palletsprojects.com/>
- 4** Jeffrey Friedl, Mastering Regular Expressions, 3rd Edition, O'Reilly Media, 2006.
- 5** RFC 791, Internet Protocol (IPv4). Available at: <https://www.rfc-editor.org/rfc/rfc791>
- 6** RFC 3986, Uniform Resource Identifier (URI): Generic Syntax. Available at: <https://www.rfc-editor.org/rfc/rfc3986>
- 7** RFC 6234, US Secure Hash Algorithms (SHA and SHA-based HMAC). Available at: <https://www.rfc-editor.org/rfc/rfc6234>